

Primers and Shortcuts: Testing Structural Preambles for LLM Graph Reasoning at Scale

Gal Barak

github.com/GalBrk

Nitzan Zacharia

github.com/NitzanZacharia

Inbal Moryles

github.com/iinbal

Abstract

Prepending a short *primer* of factual sentences about each node’s local structure — degree, clustering coefficient, a random-walk structural encoding, or connected-component membership — before a graph’s standard text encoding is an intuitively appealing way to ease graph-reasoning prompts for a large language model. Any apparent gain is illusory, however, if a solver can recover the answer directly from the primer text without ever consulting the graph, and it is confounded if the primer merely lengthens the prompt. We introduce a graph-blind shortcut solver as a content-only lower bound and a length-matched filler control as a length-only upper bound. Evaluated at $n = 40$ on qwen3-1.7b and qwen3-4b, with and without chain-of-thought, we find that no primer effect replicates across model scale. Across four arms, six tasks and six primers, several cells beat both none and filler significantly in one model size but none does so in both, and the largest consistent content effect — the degree primer on node_degree, which states the answer verbatim — reverses sign between 1.7B (+7.5 points) and 4B (−6.5). The filler is not inert: it costs the thinking arm up to 14.1 points on dense graphs and harms both sizes on edge_existence, while the largest apparent gains fall on tasks whose answer is effectively constant, where content-free text moves accuracy as much as a structural primer. Chain-of-thought on an identical checkpoint is worth +29.2 points pooled, an order of magnitude more than any primer effect measured. One cell survives every control: clustering on node_degree for qwen3-1.7b, +3.8 points, replicated at +4.3 ($p = 0.0006$).

encoded as text, independent of its underlying reasoning ability: their study varies the encoding function — adjacency lists, incident lists, and “themed” encodings renaming nodes as people or places — while holding question and graph fixed.

This work poses a complementary question: rather than altering how edges are described, does prepending a short *primer* of derived, node-level facts before the graph encoding help a model answer questions about that graph? A primer lies outside the encoding vocabulary studied by [Fatemi et al. \(2023\)](#): it is additional, redundant information computed from the graph and expressed in plain sentences. Two confounds make such an effect hard to read. If a primer helps because the model reads the answer out of the primer rather than reasoning over the graph, that is answer leakage, not reasoning. And if it helps or hurts because it lengthens the prompt, that says nothing about its *content*: [Liu et al. \(2024\)](#) and the needle-in-a-haystack protocol of [Kamradt \(2023\)](#) show that context position and length alone move retrieval accuracy, whatever the added text says.

This paper makes two contributions. First, a single, deterministic primer renderer producing seven conditions (Section 3.1) that differ only in content, so differences between them are content effects rather than formatting artifacts — among them a length-matched, information-free filler condition bounding the length term. Second, a *shortcut solver* (Section 3.2) answering each GraphQA task from rendered primer text alone, with no access to the graph, which lower-bounds what a primer gives away. We evaluate both in a density sweep on qwen3-1.7b and qwen3-4b (Section 4).

1 Introduction

[Fatemi et al. \(2023\)](#) demonstrate that a large language model’s accuracy on graph-reasoning questions depends substantially on how the graph is

2 Related Work

[Fatemi et al. \(2023\)](#) introduce GraphQA, a benchmark of graph reasoning tasks paired with a family of graph-to-text encoding functions, and show that

encoding choice, task, and the underlying graph generator all materially affect LLM accuracy. We adopt their task definitions, and their finding that the generator matters motivates fixing graph size and edge density explicitly (Section 4) rather than sampling structure incidentally.

A broader literature probes LLM graph reasoning. Wang et al. (2023) report that LLMs are brittle to spurious correlations in graph and problem structure; Guo et al. (2023) find that models struggle to exploit graph description languages directly; and Jin et al. (2024) survey the wider space of LLM-graph integration. Dziri et al. (2023) argue that transformers solve compositional tasks by linearized subgraph matching rather than systematic multi-step reasoning, which raises the question a primer’s apparent benefit also raises: is a gain evidence of reasoning, or of a shorter pattern to match against? Geirhos et al. (2020) frame this generally as *shortcut learning* — a rule that performs well on the benchmark without solving the intended problem. Our shortcut solver (Section 3.2) operationalizes that concern for the primer channel specifically, rather than for the model as a whole.

The RWSE condition adapts the random-walk structural encoding of Dwivedi et al. (2022), developed for graph transformers; we render its return probabilities as sentences rather than feeding them to an attention mechanism. The chain-of-thought arm follows Wei et al. (2022), and our paired testing follows Dror et al. (2018).

3 Method

3.1 Primers

A primer is a short preamble of factual sentences describing a graph’s local or global structure, prepended to the standard text encoding and the task question. A single renderer produces every condition, so all node-level facts share identical sentence framing (“Node N has...”) and floating-point values are rounded and formatted identically; conditions therefore differ in content alone. We evaluate seven: **none**, the encoding with no primer; **components**, one graph-level sentence giving the number of connected components; **degree**, each node’s exact degree; **clustering**, each node’s clustering coefficient; **rwse**, each node’s random-walk return probabilities (Dwivedi et al., 2022); **all**, the latter three combined; and **filler**, a content-free control (“Node N is simply present within the graph G .”) that carries no structural information at com-

parable length, and so isolates content from the confound of merely lengthening the prompt.

3.2 Shortcut solvers

For a primer’s measured accuracy to constitute evidence of graph reasoning, it must exceed what is recoverable from the primer text alone. We construct a shortcut solver taking only rendered primer text as input — never the graph object, never the full-precision statistics the primer rounds away — and answering each GraphQA task directly. It combines sixteen exact rules (deriving edge count algebraically from a `degree` primer, for instance), one heuristic rule, eight fitted rules whose fitting and scoring graph sets are kept structurally disjoint, and an exact enumeration bound for small graphs. Being structurally incapable of observing the graph, its accuracy on a (primer, task) cell is a model-independent bound: results there should be read against this *shortcut ceiling* rather than against zero, since a primer that fully determines the answer (ceiling 1.0) makes model accuracy on that cell uninformative about graph reasoning, however high.

3.3 Prompt assembly

Each prompt concatenates the primer, the graph’s text encoding, and the task question. Prompts are rebuilt from a canonicalized, parsed representation of each graph rather than reused verbatim from the source dataset, so node and edge ordering is reproducible and independent of the order the original question happened to list them in.

3.4 Significance testing

Every comparison reported here is paired: a primer condition is compared against `none` on the same graph, with the same question and gold answer, and pairs in which either arm reached the output-token cap are dropped from that comparison rather than scored as incorrect. Because a graph contributes exactly one pair to a given (task, condition) comparison, the pairs entering a single test are independent and no clustering correction is required; we do not pool across tasks, where they would not be (Dror et al., 2018).

The primary test is the exact McNemar test (McNemar, 1947), Benjamini–Hochberg corrected across the six primer conditions within a task (Benjamini and Hochberg, 1995) — $q = 0.05$, six hypotheses per family, not one family of 144. For

the comparisons we interpret substantively we additionally report a paired permutation test (sign-flipping each pair’s observed difference, 20,000 resamples with the add-one correction of North et al.) and a paired bootstrap 95% confidence interval on the effect size (20,000 resamples, whole pairs resampled together). The permutation and McNemar p -values agree to within a few thousandths on every cell where we report both, so the interval, not the second p -value, is what the extra computation buys.

4 Experimental Setup

Tasks. We evaluate on six GraphQA tasks from [Fatemi et al. \(2023\)](#): node count, edge count, node degree, connected nodes, edge existence, and cycle check.

Models. We compare `qwen3-1.7b` and `qwen3-4b` from the Qwen3 model family ([Qwen Team, 2025](#)), each evaluated in a non-thinking (greedy decoding, no chain-of-thought) and a thinking (chain-of-thought-enabled; [Wei et al., 2022](#)) variant, for four model arms in total.

Sweep design. All four arms are evaluated on one shared set of prompts, so that every model receives byte-identical inputs. Graphs are Erdős–Rényi ([Erdős and Rényi, 1959](#)) with a fixed node count of 40 and edge density swept over four levels ($p \in \{0.10, 0.20, 0.35, 0.50\}$). Every combination of the seven primer conditions (Section 3.1) and six tasks is evaluated on 100 graphs per (density, task) cell, yielding 16,800 prompts per model arm. Generation is greedy, with a raised output budget (up to 8192 tokens), since at the densest level tasks such as edge count require enumerating several hundred edges in the answer.

Auxiliary sweeps. Two comparisons in Section 5 need more power or a wider density range than the main sweep provides, and use dedicated prompt sets built by the same renderer. The first fixes `node_degree` at 400 graphs per density level over the same four densities, run twice on independently drawn samples. The second extends the range to $p \in \{0.10, \dots, 0.85\}$ on `node_degree`, where the main sweep’s ceiling at $p \leq 0.50$ leaves too little dynamic range. Both are identified explicitly wherever their numbers appear.

Metrics and reproducibility. We use exact match for integer- and boolean-valued tasks and

set-F1 for connected nodes, reporting every result against both `none` and the corresponding shortcut ceiling. Generations reaching the output-token cap are excluded from scoring rather than scored incorrect, and reported separately as a per-cell drop count. All generation is greedy, so a run is deterministic given a checkpoint and a prompt file; thinking and non-thinking arms are the same checkpoints with the model’s native reasoning channel pinned on or off, not a prompt-level instruction, so they differ in decoding mode alone. Code, prompt sets and raw generations are released with the paper.

Known validity caveats. Several cells are shortcut-dominated at this graph size rather than genuine tests of graph reasoning: any primer with one sentence per node lets node count be recovered by counting sentences; cycle check’s gold answer is “yes” for nearly every graph past the sparsest level; and degree states the node-degree answer verbatim. The one cell established as free of this contamination is node degree under `{none, components, clustering, filler}`. We report the contaminated cells with this caveat rather than omitting them, since a null or negative result on an uncontaminated cell remains informative.

5 Results

All four arms completed the sweep: 16,800 scored generations each. Unless stated otherwise, every reported Δ is a paired difference against `none` over instances where neither arm truncated, tested with an exact McNemar test ([McNemar, 1947](#)) and Benjamini–Hochberg corrected across the six primers within a task ([Benjamini and Hochberg, 1995](#)) at $q = 0.05$.

Truncation must be read before accuracy. The truncation rate is neither uniform across tasks nor across arms, and it is not ignorable: on `edge_count` the thinking arms truncate on 62–81% of instances, with individual (density, primer) cells at 100%. Because truncated generations are dropped rather than scored incorrect, the survivors there are a biased subset of the sparsest graphs, and the apparently high `none` accuracy (86.3% at 1.7B, 97.6% at 4B, against 2% in the untruncated plain arms) measures which graphs finished, not which were solved. We report `edge_count` under chain-of-thought but decline to interpret it;

several of its comparisons rest on fewer than 60 surviving pairs.

The choice to drop rather than zero-score truncated rows is itself consequential and we state it explicitly: scoring them as incorrect reverses the sign of the `rwse` effect on 1.7B `cycle_check`, because `none` truncates more often (9%) than either `clustering` or `all` (< 1%) on that task. Neither convention is uniformly more conservative.

5.1 The full density sweep

No primer effect replicates across model scale. Several (primer, task) cells significantly exceed both `none` and `filler` in one plain arm; `none` does so in both. The strongest candidate is `degree` — and `all`, which contains it — on `node_degree`, where the primer states the requested quantity verbatim: +7.5 [+3.0, +12.0] and +10.2 [+5.2, +15.2] points to `qwen3-1.7b` and +11.7 and +11.6 to `qwen3-1.7b-think` (Table 3), but -6.5 [-9.2, -4.0] and -9.2 [-12.2, -6.2] to `qwen3-4b`, with non-overlapping intervals between the sizes. A primer that *contains the answer* makes the larger model worse; 26 of its 29 `degree` losses are off by one to three, consistent with the primer competing with, rather than replacing, the model’s own enumeration of the encoding. Table 5 shows this is a ceiling artifact: `qwen3-4b` is at 99% under `none` at every density, leaving a primer no headroom and only the cost of added tokens to register, while `qwen3-1.7b` falls from 0.910 to 0.300 over the same range and does admit a positive effect.

Answer retrieval is partial even where it is complete in principle. The shortcut solver attains 1.00 on `node_degree` under `degree` and `all`, and on `edge_count` under the same two, since summing a stated degree sequence yields $2m$. No arm approaches that ceiling: the largest `edge_count` gain anywhere is `qwen3-4b` under `degree`, +27.7 points, still leaving roughly 70% of instances wrong against a bar of 100%. A primer that fully determines the answer is not a test of graph reasoning, but neither is it automatically exploited: it measures whether the model uses a fact it was handed, and largely it does not. The ceilings were fit on GraphQA’s native 5–19 node graphs; at $n=40$ the constant answers “40” and “yes” score 100% on `node_count` and

`cycle_check`, so only the exact-theorem ceilings above transfer unchanged.

Trivially-answerable tasks respond to text, not structure. The largest effects in Table 1 fall on `node_count`, whose gold answer is the constant 40. At 1.7B every condition improves on `none` (31.5%), by +2.0 to +67.2 points, in an order that follows no property of content: `clustering` +67.2, `all` +47.0, `filler` +39.8, `components` +21.2, `degree` +6.8, `rwse` +2.0 (n.s.). At 4B the six are statistically indistinguishable, `all` +8.0 to +9.8. In both arms the content-free `filler` is among the strongest. The dominant `none` failure is the answer “39”, an off-by-one from the encoder’s “nodes 0, ..., 39” preamble that any interposed text appears to disrupt. We accordingly retract the reading of this cell as a structural primer effect.

Prompt length is not a neutral covariate. The `filler` condition is not inert. It significantly harms `qwen3-1.7b` on `edge_existence` (-15.8 [-20.0, -11.8]) and `connected_nodes` (-6.2 [-10.2, -2.2]), and `qwen3-4b` on `connected_nodes` (-12.5 [-17.0, -8.0]) and `edge_existence` (-7.0 [-10.8, -3.5]), while significantly *helping* both on the trivially-answerable `node_count`. At $n=40$ it adds 1,831 characters, between `clustering` (1,631) and `rwse` (2,951; Table 6). Carrying no structural information by construction, any effect it registers is attributable to length and position alone (Liu et al., 2024; Kamradt, 2023), so comparable effects measured against `none` cannot be credited to content without it. Over a wider density range (p up to 0.85, 2,774 pairs, `qwen3-1.7b-think`) the `filler` costs -6.3 points pooled ($p < 10^{-4}$): nothing significant at $p \leq 0.35$, then -9.5, -9.3, -14.1 and -11.7 at the four densest levels. Length costs more on dense graphs than most primer contents gain anywhere here.

Chain-of-thought dominates every primer effect. Varying only the decoding mode, with checkpoint and prompts fixed (Table 7), is worth +29.2 points pooled and +50.0 at $p = 0.85$ — an order of magnitude above the largest non-retrieval primer effect measured, and consistent with Wei et al. (2022). The consequence is that the thinking arms sit at ceiling on `cycle_check`, `edge_existence` and `node_count` under every condition, and `qwen3-4b-think` on five of six tasks, leav-

Arm	Task	none	comp.	clust.	rwse	degree	filler	all
qwen3-1.7b	connected_nodes	72.5	+0.8	+1.2	+1.0	+3.5	-6.2	+2.0
	cycle_check	99.7	-0.6	+0.3	-7.5	-12.6	+0.3	-1.1
	edge_count	1.8	-1.4	-0.4	-0.8	+4.0	-0.4	+2.4
	edge_existence	70.5	-8.2	+3.8	+5.3	+4.0	-15.8	+11.5
	node_count	31.5	+21.2	+67.2	+2.0	+6.8	+39.8	+47.0
	node_degree	60.5	+1.8	+3.0	+4.0	+7.5	-0.5	+10.2
qwen3-4b	connected_nodes	86.0	+9.2	+1.0	0.0	-3.0	-12.5	-2.8
	cycle_check	99.7	-1.3	+0.3	+0.3	-1.5	-0.3	-0.5
	edge_count	2.0	-1.3	-1.3	-1.5	+27.7	+1.2	+3.8
	edge_existence	90.5	-2.8	+4.0	-10.0	-4.3	-7.0	-9.8
	node_count	90.2	+9.0	+8.8	+9.2	+8.0	+9.2	+9.8
	node_degree	99.2	+0.2	-0.5	-2.2	-6.5	-0.5	-9.2

Table 1: Main sweep, plain arms (thinking arms in Table 3), $n=40$, pooled over densities $p \in \{0.10, 0.20, 0.35, 0.50\}$, 100 graphs per (task, primer, density) cell. The *none* column is absolute accuracy (%); all other columns are paired differences against *none* in percentage points. Exact match throughout, including *connected_nodes*, whose set-F1 (Section 4) sits at 96–100% for every condition and conceals the differences exact match resolves. Truncated generations are excluded pairwise, giving $n \in [391, 400]$ paired instances per cell. **Bold** indicates significance under an exact McNemar test after Benjamini–Hochberg correction across the six primers within a task ($q = 0.05$).

ing no headroom for a primer effect. Primers do change how often those arms fail to terminate — a process effect, not an accuracy one: for *qwen3-1.7b-think* *degree* truncates 3.4 points more often than *none* ($p < 0.001$) while *all*, *filler* and *clustering* truncate 1.3–3.4 points less; for *qwen3-4b-think* every primer truncates 2.0 to 4.3 points less (all $p < 0.001$).

The one cell that survives every control. One cell clears every control, at four times the power of any cell in Table 1. On a dedicated sweep of 400 graphs per density level, *clustering* improves *qwen3-1.7b* on *node_degree* by +3.8 points [+1.5, +6.2], and an independent sample reproduces it at +4.3 [+1.9, +6.6] ($p = 0.0017$ and 0.0006 ; Table 2). It is positive at every density level and attributable neither to answer leakage (ceiling 0.08) nor to length (*clustering* is shorter than *filler*, which does not move this task). The main sweep is consistent but underpowered here, measuring +3.0 ($p = 0.256$) at 100 graphs per cell. The effect does not survive into the thinking arm (+1.0, $p = 0.694$), and *qwen3-4b* cannot test it at all, being saturated under *none*: a replicated but model- and mode-specific effect, not a general property of clustering primers.

6 Discussion and Conclusion

The central negative result is a failure to replicate across scale: of the 144 (arm, task, primer) comparisons, no primer effect clearing both *none* and

filler in one model size does so in the other. Three regularities account for most of the table, and *none* is the hypothesized mechanism.

First, a primer helps reliably only where it states the requested quantity verbatim, and then only partially: *degree* and *all* move *node_degree* and *edge_count* and nothing else consistently, while the statistics naming no answer survive both controls in no more than one arm. This is answer retrieval done badly — never more than 28 of 100 available points, and at 4B backwards. Second, added text is an active treatment regardless of content, harming both sizes on *edge_existence* and *connected_nodes* and helping both on *node_count*, at magnitudes (−15.8 to +39.8) exceeding nearly every content effect here; a design comparing a primer only against *none* therefore estimates content and length confounded, with the length term often the larger. Third, the headroom is largely consumed by the decoding mode: chain-of-thought is worth +29.2 points pooled and moves most cells to ceiling, and where it does not saturate a task it censors it, the 62–81% truncation on *edge_count* removing that task from both thinking arms.

The interesting quantity is therefore not whether a structural preamble helps, but where it *could*: of the 24 (arm, task) cells, 10 sit at or above 98% under *none*, 2 are censored, and 2 more have constant gold answers, and the few that remain are exactly where the replicated *clustering* effect was found.

Limitations

This sweep covers only two model sizes (1.7B and 4B parameters) from a single model family, so the findings may not generalize to substantially larger models or to other architectures. Several (primer condition, task) cells are known in advance to be shortcut-dominated at this graph size and density range (Section 4); results on those cells bound what can be concluded about genuine graph reasoning even though we report them. Graphs are drawn from a single generator family (Erdős–Rényi); whether primer effects transfer to other graph generators, which Fatemi et al. (2023) show matters for encoding effects, remains untested here, and, as noted in Section 2, we found directly that varying the generator family itself is not a safe way to test this, since doing so confounds structure with task difficulty. Compute and time constraints limited this sweep to a single node-count/density design rather than a broader sweep over graph sizes.

Finally, the length control is a bracket rather than an exact match: the `filler` condition is longer than `degree` and `clustering` and shorter than `rwse` and `all` (Table 6), so it bounds the length term without eliminating it. A design holding rendered length exactly constant while varying structure would isolate content more sharply than subtracting a bracketing control, and we regard that as the natural next step.

Ethics Statement

This work evaluates open-weight language models on synthetically generated Erdős–Rényi graphs. It involves no human subjects, no personal or identifying data, and no annotation labour: gold answers are computed programmatically from the graphs rather than collected from people. We see no direct risk of harm arising from the findings, which are a negative result about a prompting technique. The principal societal cost is the compute expended, reported in Section 4.

References

- Yoav Benjamini and Yosef Hochberg. 1995. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1):289–300.
- Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. 2018. The hitchhiker’s guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1383–1392.
- Vijay Prakash Dwivedi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. 2022. Graph neural networks with learnable structural and positional representations. In *International Conference on Learning Representations (ICLR)*.
- Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D. Hwang, Soumya Sanyal, Sean Welleck, Xiang Ren, Allyson Ettinger, Zaid Harchaoui, and Yejin Choi. 2023. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.
- Paul Erdős and Alfréd Rényi. 1959. On random graphs i. *Publicationes Mathematicae Debrecen*, 6:290–297.
- Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2023. Talk like a graph: Encoding graphs for large language models. *arXiv preprint arXiv:2310.04560*.
- Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. 2020. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673.
- Jiayan Guo, Lun Du, and Hengyu Liu. 2023. Gpt4graph: Can large language models understand graph structured data? an empirical evaluation and benchmarking. *arXiv preprint arXiv:2305.15066*.
- Bowen Jin, Gang Liu, Chi Han, Meng Jiang, Heng Ji, and Jiawei Han. 2024. Large language models on graphs: A comprehensive survey. *IEEE Transactions on Knowledge and Data Engineering*, 36(12):8622–8642.
- Greg Kamradt. 2023. Needle in a haystack – pressure testing llms. https://github.com/gkamradt/LLMTest_NeedleInAHaystack.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.
- Qwen Team. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. 2023. Can language models solve graph problems in natural language? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.

Sample	Δ	n	95% CI	p
Original	+3.8	1600	[+1.5, +6.2]	0.0017
Replication	+4.3	1600	[+1.9, +6.6]	0.0006

Table 2: qwen3-1.7b, node_degree, clustering against none, $n=40$, pooled over $p \in \{0.10, 0.20, 0.35, 0.50\}$ with 400 paired instances per level. Δ in percentage points; n is paired instances, the interval is a paired bootstrap on the effect size, and p is the exact McNemar test (permutation p : 0.0015 and 0.0007). The replication draws a fresh graph sample at the same design points. The shortcut ceiling for this cell is 0.08, so the effect is not answer retrieval, and clustering is 200 characters *shorter* than the filler control (Table 6), which registers no effect on this task in either 1.7B arm (-0.5 and -3.5 , both n.s.).

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35.

A Supplementary tables

Arm	Task	none	comp.	clust.	rwse	degree	filler	all
qwen3-1.7b-think	connected_nodes	79.7	+2.8	+1.5	-0.3	+7.4	+2.0	+6.1
	cycle_check	100.0	0.0	0.0	0.0	-0.7	0.0	0.0
	edge_count [†]	86.3	+7.4	-8.7	-4.0	-61.5	+3.4	-8.3
	edge_existence	99.2	-0.8	-1.0	-1.3	-2.3	-0.8	-2.0
	node_count	98.6	+0.9	+1.4	+1.4	+0.8	+1.4	+1.4
	node_degree	76.8	-1.0	+1.0	+2.5	+11.7	-3.5	+11.6
qwen3-4b-think	connected_nodes	98.7	+0.5	0.0	0.0	0.0	+0.8	-0.5
	cycle_check	100.0	0.0	0.0	0.0	-0.8	0.0	0.0
	edge_count [†]	97.6	+1.7	+1.7	0.0	-23.5	-2.0	-41.5
	edge_existence	100.0	0.0	0.0	0.0	0.0	0.0	0.0
	node_count	99.5	0.0	+0.5	+0.5	+0.5	+0.5	+0.5
	node_degree	99.7	-0.3	-0.3	-2.1	0.0	0.0	-0.8

Table 3: Main sweep, thinking arms; conventions as in Table 1. Both arms are at or near ceiling on most tasks, leaving no headroom for a primer effect. [†]Censored beyond interpretation (Table 4); $n \in [13, 60]$ surviving pairs.

Task	1.7B		4B	
	plain	think	plain	think
edge_count	22.4	81.0	0.6	62.3
cycle_check	6.1	6.5	1.0	1.9
all others (max)	0.8	2.5	0.1	3.5
Overall	4.8	15.4	0.3	11.7

Table 4: Truncation rate (% of generations reaching the 8192-token output cap), by task and model arm. edge_count under chain-of-thought is censored past the point of interpretability: several (density, primer) cells are 100% truncated, and the generations that terminate are the sparse, easy graphs.

Condition	Primer (chars)	Prompt (chars)
none	0	2996
components	39	3036
degree	897	3894
clustering	1631	4628
filler	1831	4828
rwse	2951	5948
all	4697	7694

Table 6: Median rendered length by condition at $n=40$ (node_degree, 400 prompts per condition). The content-free filler control is longer than degree and clustering and shorter than rwse and all, so it brackets the informative primers rather than matching any single one exactly.

Condition	density p			
	0.10	0.20	0.35	0.50
<i>qwen3-1.7b</i>				
none	0.910	0.800	0.410	0.300
clustering	0.930	0.820	0.470	0.320
degree	0.940	0.900	0.590	0.290
filler	0.960	0.730	0.410	0.300
all	0.970	0.900	0.580	0.380
<i>qwen3-4b</i>				
none	1.000	0.990	0.990	0.990
clustering	1.000	1.000	0.990	0.960
degree	0.940	0.980	0.920	0.870
filler	1.000	0.990	0.990	0.970
all	0.880	0.960	0.920	0.840

Table 5: node_degree accuracy by density, plain arms, selected conditions. The 1.7B arm degrades steeply with density and admits a primer effect; the 4B arm is saturated under none at every density, so the only movement available to a primer is downward, and degree and all — the two primers that state the answer — supply it.

p	n	plain	think	Δ
0.10	1185	0.929	0.958	+2.9
0.20	1196	0.781	0.901	+12.0
0.35	1199	0.425	0.655	+22.9
0.50	1200	0.292	0.581	+28.8
0.65	1552	0.126	0.503	+37.7
0.75	1554	0.094	0.476	+38.2
0.85	1553	0.054	0.554	+50.0
pooled	9439	0.318	0.610	+29.2

Table 7: Chain-of-thought against greedy decoding on the identical qwen3-1.7b checkpoint and byte-identical prompts, node_degree, pooled over the none, components, clustering and degree conditions. Δ in percentage points; every level is significant at $p < 10^{-3}$ under an exact McNemar test, the pooled comparison beyond the range of exact computation ($z = 46.4$).